

Benchmark RADAR: Living Search Engine for Retrieval and Discovery of AI Benchmark Research

Koutian Wu^{1,2 *} Junjie Zhou⁴ Ergan Shang³ Jiayu Wang⁵ Pengqian Han⁶

¹ Earth and Space AI ² Tacite AI ³ Carnegie Mellon University

⁴ Hangzhou Dianzi University ⁵ Xi'an Jiaotong University

⁶ The University of Auckland

Abstract

Researchers choosing an AI benchmark need to find its tasks, released artifacts, and reported model results. Benchmark Radar brings this evidence into one searchable catalog while preserving each source record and its citations. At the 2026-09-07 cutoff, the catalog contains 1,283 benchmark records across 4 sources, with 12,916 numeric score observations on 790 records. The remaining 493 records have no numeric score; 464 of them still provide a paper, repository, or dataset link. A score-only catalog would lose these routes to prior work. Across the full catalog, we distinguish models with reported scores from cited documents and measure the availability of score scales and dates. Missing metadata limits comparisons without removing the underlying evidence. A contributor's prior-art search illustrates how to inspect candidates before selecting evaluations. Readers can access the same benchmark records through the dashboard, downloads, and offline search clients.

1,283 records. 790 with scores. 493 without.

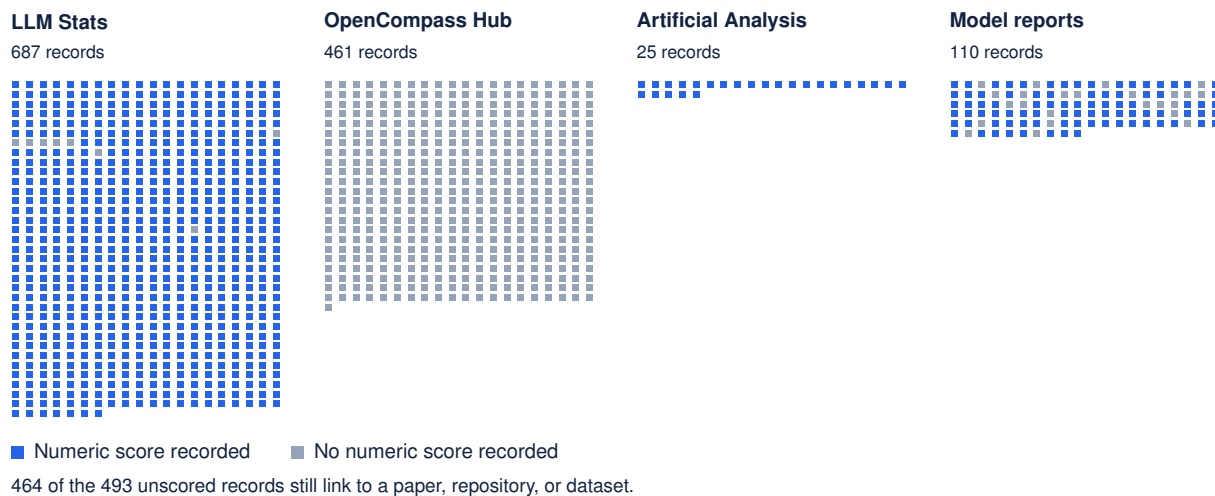


Figure 1: The complete catalog: one dot per source record, including records without scores. Dots link to benchmark details. Source labels identify provenance; the layout applies no date or score filter.

**Corresponding author: k@tacite.ai

1 Introduction

Before designing a new evaluation, a researcher needs to know which benchmarks already test the proposed capability, whether their data and code are available, and which models have been evaluated on them. Answering these questions requires following evidence across paper servers, code repositories, dataset hubs, leaderboards, and model reports. A benchmark name alone rarely supplies the test version, released artifacts, and evaluation settings needed to judge whether it fits the research question.

The range of evaluation tasks makes this search difficult. MMLU [6], GPQA [22], and Humanity’s Last Exam [20, 26] test knowledge and reasoning. SWE-bench tests issue resolution [11]; LiveCodeBench uses date-based code-generation tests [10]; and Terminal-Bench and Long-Horizon-Terminal-Bench evaluate command-line agents [14, 16]. Researchers must also find domain-specific tests, including FinanceBench [9], MATH [7], SciBench [24], LAB-Bench [13], and ChemBench [17]. Choosing among them requires inspecting tasks, scoring rules, and released artifacts.

LLM Stats, OpenCompass, and Artificial Analysis provide benchmark catalogs, evaluation platforms, and model comparisons [1, 15, 19]. Benchmark Radar builds on their records by adding daily discovery and model-report evidence to a shared search interface. A reader can retrieve a benchmark, follow its paper and artifact links, and inspect which reports mention it and under which settings they report scores. Each source record keeps its identity and citations so the reader can trace a result back to the evidence.

We examine what researchers can learn from the complete catalog, including records with missing scores or citations. The census separates benchmark coverage, model coverage, and document coverage, then identifies which measurements support comparisons. The central finding is practical: 464 records with no numeric score still offer links for a prior-art search. Restricting discovery to scored evaluations would hide them. Appendix B follows a contributor who used Benchmark Radar to investigate a proposed evaluation.¹

2 Methodology

2.1 One Catalog and Its Reader Interfaces

Benchmark Radar adapts BuilderPulse’s daily public-source collection approach [25] to benchmark discovery. Four sources contribute benchmark records to a common catalog: LLM Stats, OpenCompass Hub, Artificial Analysis, and Model reports. The normalization step preserves one record per source benchmark, its numeric observations, model identities, documents, and citations. Model reports include model cards, technical reports, system cards, and release posts [18]. Document types describe the evidence available for a record.

A benchmark belongs in the catalog even if it has no score, known date, or cited document. Reviewed identity links connect related records without combining their scores or transferring model and document counts. The same benchmark IDs reach web search, detail pages, offline clients, and the exported catalog. Table 1 summarizes the reader interfaces.

Benchmark Frontier, on the Leaderboard tab, requires a numeric score and excludes known pre-2024 benchmarks. It retains scored records with unknown dates or counts through labelled marks, and keeps unverified scales outside Pareto calculations. Its initial score cutoff is 70. The linked score ranking retains its date and numeric-score requirements independently of that cutoff. General catalog search and exports retain the full population. The census in this paper applies none of these display filters.

¹Source code: <https://github.com/ktwu01/benchmark-radar>.

Interface	Reader question	Evidence and scope
Today	What appeared or changed?	Daily discovery observations, source health, and links.
Search	Which benchmarks match this task?	The complete catalog, including unscored records.
Leaderboard	Which evaluations combine lower scores with broader measured use?	Scored source records; distinct scored models or cited documents supply the selected count.
Saturation	What scores and settings were reported?	Benchmark browsing and complete score histories; search reaches every source and year.
CLI and HTTP	Can I inspect the same evidence locally?	One query service and the same benchmark IDs and response format.

Table 1: Reader questions and evidence scope. A search result is a candidate for inspection, not a suitability judgment.

2.2 Discovery Evidence and Source Health

Daily discovery observations describe mentions, releases, and updates found across public sources. An observation is one collected record; an artifact is a paper, repository, dataset, release, or page linked by exact identifiers. These objects differ from source-specific benchmark records. We retain their histories alongside the catalog without adding discovery observations to the benchmark total.

Each collection run searches a 48-hour window, records counts and errors by source, removes future-dated rows, and requires healthy core sources before publication. At the cutoff, arXiv [2], Hugging Face Hub [8], and GitHub Search [5] were the core sources. Additional routes cover scholarly indexes, dataset hosts, repository releases, and institutional feeds. Appendix A records their cutoff status.

The 2026-09-07 snapshot records 1,003 fetched rows and 954 candidates after duplicate removal. Of these, 366 qualified for publication and 138 met the recommendation threshold. These are discovery-run counts, separate from the benchmark catalog census. Published evidence retains source URLs, retrieval times, parser versions, identifiers, and payload checksums. Raw responses and credentials remain outside the public artifact.

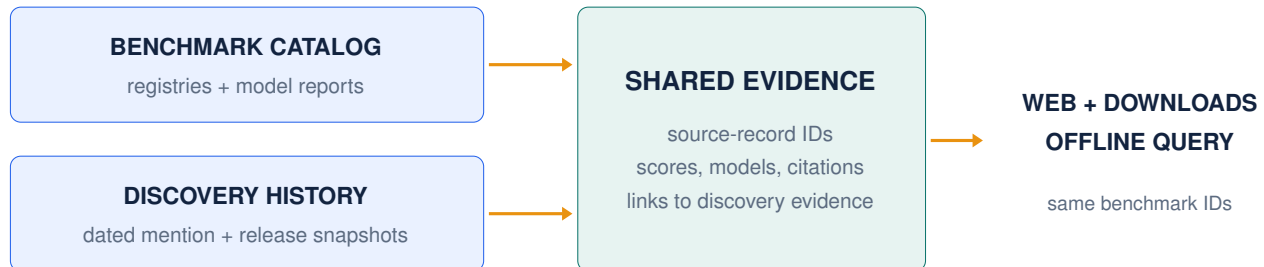


Figure 2: Two input paths support publication: dated discovery snapshots preserve mention and release evidence, while benchmark registries and model reports populate the shared catalog. Clients use the catalog’s stable source-record IDs.

2.3 Identity and Search

The resolver links discovery observations that share a DOI, arXiv identifier, OpenReview identifier, GitHub repository, or Hugging Face artifact. Similar titles alone do not force a merge. Catalog identity review also preserves each source record; a link to another record does not establish equivalent test versions or protocols.

Lexical search retrieves candidates across all 1,283 records. BM25F supplies the primary retrieval score, with bounded boosts for name and phrase matches. Results expose matched and missing query tokens, fields, coverage, and score components. Source membership does not change ranking. The shared query service supplies CLI and HTTP responses with local data provenance and retrieval mode. Installed clients search offline and update through explicit initialization or synchronization.

A separate priority score for daily recommendations combines relevance (35%), evidence (20%), recency (20%), and adoption (25%) on a 0–100 scale. These weights describe reading priority for discovery observations. An agent assessing benchmark suitability can try focused query variants, inspect record details, and read the cited sources; that judgment occurs after candidate retrieval.

2.4 Measurement Rules for the Full-Catalog Census

We start with every record in the rebuilt `benchmark-index.json` and read its own detail shard. We count finite numeric score observations once by observation ID. For each benchmark record, we count distinct scored models by source model ID, preserving separately evaluated configurations, and cited documents by document ID. Repeated scores or documents for a model do not increase that record’s model count. Missing model identities or absent measurements remain unknown. The global model registry uses its recorded identity links; we do not sum per-benchmark model counts to obtain a global total.

We apply score-scale and date requirements to evidence fields across all sources. A declared percentage unit, known score direction, and values within 0–100 support a percentage-scale summary. A display rescaling or an aggregator’s declared maximum alone does not verify a percentage metric. Even a valid percentage scale does not establish matching instruments, prompts, tool access, attempts, or evaluators [4, 18].

For date coverage, we count recorded benchmark release dates without substituting crawl times or model release dates. Score entries carry their date basis separately. A model’s announcement date can provide a labelled timeline proxy; it cannot establish when a benchmark score was measured or published. Missing metadata restricts the calculation that needs it while leaving the source record and numeric observations inspectable.

3 Results

3.1 Full Catalog Coverage

The rebuilt catalog contains 1,283 source records across 4 sources. We found 12,916 numeric score observations on 790 records, with 493 records lacking numeric scores (Table 2). These are source-record counts: two sources can describe a related benchmark, and we retain both records without claiming they represent distinct underlying tests.

The sources differ in the evidence their snapshots contain. LLM Stats supplies benchmark descriptions and reported results [15]. OpenCompass Hub supplies registry metadata and artifact links [19]; its snapshot contains no model-score observations. Artificial Analysis supplies evaluation records

Source	Records	Scored	Unscored	Numeric scores
LLM Stats	687	679	8	5,544
OpenCompass Hub	461	0	461	0
Artificial Analysis	25	25	0	7,050
Model reports	110	86	24	322
Total	1,283	790	493	12,916

Table 2: A census of the full catalog. Scored and unscored records reconcile to the total within each source. Numeric scores count observations, not models or documents.

and scores [1]. Model reports contribute mentions, scores, and recorded evaluation settings. These differences describe the collected evidence, without establishing a preferred source tier.

Of the 493 unscored records, 464 have at least one paper, repository, or dataset link. Across the full catalog, 475 records have a paper link, 506 have a repository link, and 293 have a dataset link; the categories overlap. For example, the OpenCompass Hub record for A-Bench links a paper, code repository, and dataset despite having no archived score. These links give researchers routes to inspect tasks and baselines before deciding whether an evaluation fits their work. Link presence does not establish current availability or permission to use an artifact.

3.2 Models and Documents Measure Different Kinds of Coverage

The common document registry contains 1,208 distinct cited documents, attached to 1,278 of the 1,283 benchmark records. Five records have no cited documents. The shared model registry contains 868 model identities. Within the catalog, all 790 scored records have sufficient model IDs to count distinct models; unscored records have unknown scored-model coverage.

Table 3 illustrates why these counts cannot substitute for one another. The Artificial Analysis record for GPQA Diamond contains scores for 586 models and cites one registry page. The Model reports record cites 27 documents and contains 21 numeric observations for 19 models. Both describe useful evidence, but adding their counts would require reviewed model and evaluation identities. Neither row alone measures adoption across the field.

Benchmark record	Source	Models	Documents	Scores
GPQA Diamond	Artificial Analysis	586	1	586
GPQA Diamond	Model reports	19	27	21
Humanity’s Last Exam	Artificial Analysis	577	1	577
SciCode	Artificial Analysis	577	1	577
CritPt	Artificial Analysis	492	1	492

Table 3: Selected source records illustrating model and document counts; this is not a ranking. The machine-readable census retains all records and the cited document IDs for each row.

The HLE, SciCode, and CritPt examples show model coverage that a report-only count would miss. Repeated scores and repeated citations remain useful evidence, but they do not create additional distinct models on a record. Conversely, one registry page can document hundreds of evaluated models.

3.3 Coverage Required for Score and Time Comparisons

Only 82 of the 790 scored records declare a percentage unit with a known direction and values within 0–100. The remaining 708 scored records use other or unverified scales. We retain their numeric observations, which do not support a shared percentage-headroom calculation. Table 4 accounts for the entire catalog before any such comparison.

Measurement state	Records	Interpretation
Numeric scores, declared percentage scale	82	Supports scale-specific summaries; protocols still need checking.
Numeric scores, other or unverified scale	708	Retain scores; do not assume a 100-point ceiling.
No numeric score	493	Score headroom remains unknown.
Full population	1,283	Every source record remains in the census.

Table 4: Measurement eligibility, using the same rule across sources. A declared scale permits a calculation on that scale; it does not prove that evaluations share a protocol.

The catalog records benchmark release dates for 615 records, leaving 668 without a release date. Across score observations, 12,594 entries carry model-announcement dates and 322 carry document-publication dates. Neither date basis directly establishes an evaluation date. This archive therefore cannot support a catalog-wide claim about score stagnation from a gap between model-report mentions and score-entry dates. Researchers must first establish dates and comparable evaluation settings for the relevant source records.

A small gap to a metric ceiling can describe a reported setup. It does not show that a benchmark has exhausted its ability to distinguish models across versions and settings. Catalog-wide headroom or recency estimates require the eligible measurement coverage alongside the statistic.²

3.4 Discovery Coverage Alongside the Catalog

The daily discovery collection contains 11,068 observations and 6,546 artifacts linked by exact identifiers across 46 snapshots. Four snapshots are simulated historical backfills; their dates describe reconstructed collection windows. These discovery units remain separate from benchmark records.

Five discovery source labels account for 9,743 of 11,068 observations (88.0%). Figure 3 includes the remaining labels in one aggregate bar. Source caps and collection failures can affect this mix, so it does not estimate the distribution of benchmark research. Fifty-nine of 6,546 artifacts have observations from multiple sources; missing or inconsistent identifiers can leave further relationships unresolved.

4 Limitations and Future Work

Lexical search can miss paraphrased benchmark names, renamed tasks, or descriptions of related ideas. Semantic retrieval could broaden the candidate set while preserving source links and retrieval explanations [21]. The worked example combines catalog queries with web search; evaluating retrieval quality requires relevance judgments over a broader set of queries and candidate records.

The catalog reflects dated inputs with different collection scopes. Registry snapshots and model reports do not cover the same dates or evaluation settings. Public endpoint limits, failed requests,

²Junjie Zhou contributed a score-archive audit and earlier report revisions, documented in [issue #457](#).

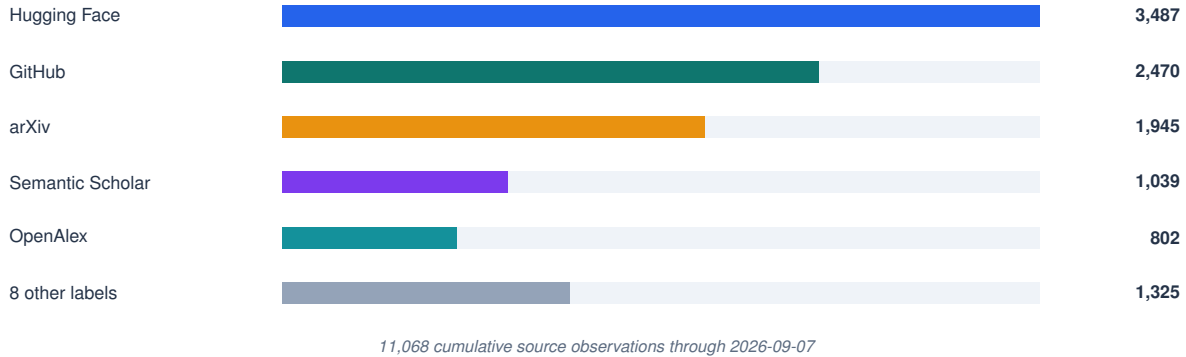


Figure 3: Discovery observations by source through 2026-09-07. Repeat sightings count as observations; these bars do not count benchmark records.

missing identifiers, and incomplete metadata affect coverage. We retain scores with unknown protocols and scales so readers can inspect them, but we do not treat them as comparable measurements. Matching archive labels alone cannot verify independent repeated runs. Reviewing primary sources for test versions, tools, attempts, evaluators, and score-publication dates remains necessary.

The capability-classification component, KW-Bench, produces 5,863 unclassified tracks in this rebuild. A classification track represents an artifact or a declared task within a benchmark suite, drawn from discovery evidence. The deterministic null extractor used in CI assigns no capability levels, so this run does not validate capability classification. These tracks do not change the benchmark catalog count.

Our quantitative results describe evidence coverage in this catalog. They do not estimate field-wide benchmark adoption, establish saturation, or measure retrieval precision. The worked example records one contributor’s workflow and has no controlled baseline. Broader retrieval evaluation requires reviewed relevance judgments, including results outside familiar benchmark families.

5 Conclusion

Benchmark Radar gives researchers access to 1,283 source-preserving benchmark records and 12,916 numeric score observations. The full census shows why discovery must include missing measurements: 464 unscored records still link to material that researchers can inspect. It also separates model coverage from document coverage and identifies the scale and date evidence needed for comparisons. Researchers can use these records to assemble prior art, inspect evaluation settings, and select evidence for further review. The next priorities are retrieval evaluation with reviewed labels and better measurement metadata across sources.

Author note. Koutian Wu led the work and manuscript preparation, built the initial collection pipeline, and implemented data aggregation across sources. Ergan Shang prepared figures and contributed copyediting. Pengqian Han contributed copyediting.

References

- [1] Artificial Analysis. Artificial analysis intelligence benchmarking methodology, 2026. URL <https://artificialanalysis.ai/methodology/intelligence-benchmarking>. Accessed 2026-09-06.

- [2] arXiv. arXiv API User Manual, 2026. URL <https://info.arxiv.org/help/api/user-manual.html>. Accessed 2026-09-06.
- [3] Citation File Format developers. Citation file format 1.2.0, 2021. URL <https://citation-file-format.github.io/>.
- [4] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.
- [5] GitHub Docs. REST API Endpoints for Search, 2026. URL <https://docs.github.com/en/rest/search/search>. Accessed 2026-09-06.
- [6] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *International Conference on Learning Representations*, 2021. URL <https://arxiv.org/abs/2009.03300>.
- [7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- [8] Hugging Face. Hugging Face Hub Documentation, 2026. URL <https://huggingface.co/docs/hub/index>. Accessed 2026-09-06.
- [9] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023. URL <https://arxiv.org/abs/2311.11944>.
- [10] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. URL <https://arxiv.org/abs/2403.07974>.
- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [12] Daniel S. Katz et al. Recognizing the value of software: a software citation guide. *F1000Research*, 9:1257, 2021. doi: 10.12688/f1000research.26932.2. URL <https://doi.org/10.12688/f1000research.26932.2>.
- [13] Jon M. Laurent, Joseph D. Janizek, Michael Ruzo, Michaela M. Hinks, Michael J. Hammerling, Siddharth Narayanan, Manvitha Ponnampati, Andrew D. White, and Samuel G. Rodrigues. LAB-Bench: Measuring capabilities of language models for biology research. *arXiv preprint arXiv:2407.10362*, 2024. URL <https://arxiv.org/abs/2407.10362>.
- [14] Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang, Junyao Yang, Zhichao Liu, Xiyang Wu, Anhao Li, Yue Yu, Ninghao Liu, et al. Long-horizon-terminal-bench: Testing the limits of agents on long-horizon terminal tasks with dense reward-based grading. *arXiv preprint arXiv:2607.08964*, 2026.
- [15] LLM Stats. AI and LLM benchmarks 2026: Rankings, scores and results, 2026. URL <https://llm-stats.com/benchmarks/>. Accessed 2026-09-06.
- [16] Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- [17] Adrian Mirza et al. Are large language models superhuman chemists? *Nature Chemistry*, 2025. doi: 10.1038/s41557-025-01788-5. URL <https://doi.org/10.1038/s41557-025-01788-5>.

- [18] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.
- [19] OpenCompass Contributors. Dataset statistics, 2026. URL https://opencompass.readthedocs.io/en/latest/dataset_statistics.html. Accessed 2026-09-06.
- [20] Long Phan et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025. URL <https://arxiv.org/abs/2501.14249>.
- [21] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410.
- [22] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023. URL <https://arxiv.org/abs/2311.12022>.
- [23] Arfon M. Smith, Daniel S. Katz, and Kyle E. Niemeyer. Software citation principles. *PeerJ Computer Science*, 2:e86, 2016. doi: 10.7717/peerj-cs.86. URL <https://doi.org/10.7717/peerj-cs.86>.
- [24] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. SciBench: Evaluating college-level scientific problem-solving abilities of large language models. *arXiv preprint arXiv:2307.10635*, 2023. URL <https://arxiv.org/abs/2307.10635>.
- [25] L. Xiaopai. Builderpulse: Ai-powered daily intelligence for indie hackers and builders. GitHub, 2026. URL <https://github.com/BuilderPulse/BuilderPulse>.
- [26] Junwei Zhou, Zhen Sun, Binyu Li, Jiangyu Zhou, Yuexi Pan, Hengyu Wang, Honghe Ren, Xiaohan Jia, Xueyang Zhou, Xiaoyu Cao, et al. Asi-bench: At the dawn of artificial superintelligence. *arXiv preprint arXiv:2608.17271*, 2026.

Appendix

A Source Inventory and Health

Table 5 lists source connectors that completed successfully at the cutoff. Row counts are measured before duplicate removal and ranking. A connector can be healthy with zero eligible rows if its request succeeds and the response can be parsed. Core sources cover papers, shared model or dataset artifacts, and code; all must be healthy before publication. Failures in optional sources are reported but do not block publication.

Connector	Role	Cutoff status	Core
arXiv	Primary papers via selected AI, language, vision, and software categories	Healthy; 46 rows	Yes
Hugging Face Hub	Datasets and Spaces connected to benchmark artifacts	Healthy; 101 rows	Yes
GitHub Search	Code repositories and benchmark artifacts	Healthy; 300 rows; at cap	Yes
GitHub Organizations	Reviewed organization repositories	Healthy; 6 rows	No
Hugging Face Papers	Community-surfaced papers	Healthy; 26 rows	No
Kaggle Datasets	Public benchmark datasets	Healthy; 30 rows	No
Zenodo	DOI-bearing research artifacts	Healthy; 65 rows	No
Crossref	DOI metadata	Healthy; 180 rows	No
OpenReview	Conference submissions	Healthy; no eligible rows	No
GitHub Releases	First-party releases	Healthy; 1 row	No
OpenAlex	Scholarly discovery metadata	Healthy; 248 rows	No

Table 5: Healthy direct discovery connectors at the current cutoff.

Table 6 lists research and engineering feeds operated by the institutions being monitored. They can surface benchmark papers, datasets, model announcements, and engineering posts. The feed collector returned no records at the cutoff. Semantic Scholar returned a malformed payload, and Brave Search lacked an API key. These optional-route failures remain recorded in the snapshot. Searches restricted to an organization’s website cover institutions that lack a verified feed.

Feeds 1-12	Feeds 13-24
Meituan Engineering	Ai2
OpenAI News	Together AI
Google AI	Sakana AI
Google DeepMind	Qwen
Google Research	Ollama
Apple Machine Learning Research	Stability AI
AWS Machine Learning	Nomic AI
Hugging Face Blog	Replicate
Microsoft Research	NVIDIA Developer
NVIDIA AI Blog	IBM Research
Mistral AI	Databricks
Meta Research	LangChain

Table 6: First-party research and engineering feeds monitored by Benchmark Radar.

B Worked Use Case: Prior-Art Check for a New Evaluation

The contributor wanted to determine whether a proposed evaluation duplicated existing work. During August 2026, they surveyed recent work on credit assignment in agentic training, using small Qwen-series models as a requirement for reproducible baseline experiments. The search produced a focused comparison table that the contributor used to assess the proposed design.

The contributor gave the task to a coding agent together with the public setup instructions for Benchmark Radar. The agent installed the command-line client and the Benchmark Radar Skill, a set of instructions for using the client. It then downloaded the local corpus and queried candidate records offline.

```

what can i do with benchmark-radar cli?

+ Thought: 328ms

$ benchmark-radar --help
usage: benchmark-radar [-h] [--author-output AUTHOR_OUTPUT]
                        [--author-contacts AUTHOR_CONTACTS]
                        [--author-repo-limit AUTHOR_REPO_LIMIT]
                        [--author-contributors AUTHOR_CONTRIBUTORS]
                        [--author-emails] [--config CONFIG] [--output OUTPUT]
                        [--json-output JSON_OUTPUT]
                        [--social-output SOCIAL_OUTPUT] [--items ITEMS]
                        [--repo REPO] [--channels CHANNELS]
                        [--existing-body EXISTING_BODY] [--since SINCE]
                        [--until UNTIL] [--snapshot-dir SNAPSHOT_DIR]
--
Click to expand

+ Thought: 531ms

The CLI has two groups:

Consumer (query) commands – for searching and inspecting local benchmark data:
- search "query" --scope catalog|radar|all --json – discover benchmarks (catalog = verified records, radar = recent unverified evidence like papers/repos/datasets)
- show "key-or-slug" --json – inspect one record in detail
- recent --json – newest Radar evidence
- status --json – local data health/provenance
- init / sync – set up or refresh the local data
- serve --host 127.0.0.1 --port 8765 – local HTTP interface for the same queries

Maintainer commands – for operating the radar pipeline (not for ordinary use): run, rebuild, backfill, migrate, rescore, simulate-history, export, classify, authors, social, normalize-external, build-data-release.

For your local benchmark search use case, you'll mostly need search, show, recent, status, and serve.

Build · DeepSeek V4 Flash DeepSeek
..... esc Interrupt 150.1K (15%) · $0.03 ctrl+p commands

```

Figure 4: A coding agent installs the Benchmark Radar client and its usage instructions, then runs local queries.

The agent inspected paper, repository, and dataset links to decide which sources to open next. Figures 5 and 6 show two retrieved records: one has a repository link, and the other has both paper and repository links. Neither records a dataset link, leaving dataset availability for a follow-up check.

```

    "rank": 18,
    "recommended": false,
    "score": 18.75,
    "slug": null,
    "snapshot_date": "2026-09-01",
    "source": "Hugging Face Papers",
    "source_id": "2608.28476",
    "updated_at": "2026-08-28T00:00:00+00:00",
    "url": "https://huggingface.co/papers/2608.28476"
  },
  {
    "categories": [
      "benchmark"
    ],
    "description": "LTABC-KM is a long-tail-aware adaptive artificial bee colony-K-means for unsupervised image clustering. It leverages density-aware coreset and graph-propagation assignment to recover minority tail modes without labels, achieving significant Macro-F1, NMI and ARI improvements on imbalanced benchmarks with favorable quality-computation trade-offs.",
    "has_dataset": false,
    "has_paper": false,
    "has_repo": true,
    "has_size": false,
    "key": "radar:github:Lutra11/LTABC-KM",
    "kind": "radar",
    "languages": [],
    "match": {
      "idf_coverage": 0.4391,
      "matched_fields": [
        "description"
      ],
      "matched_tokens": [
        "assignment"
      ],
      "missing_tokens": [
        "credit",
        "training"
      ],
      "phrase_fields": [],
      "query_coverage": 0.3333,
      "ranking_algorithm": "bm25f_v3",
      "reason": "1 of 3 unique query tokens matched across fields; IDF coverage 0.44",

```

"has_dataset": false,
 "has_paper": false,
 "has_repo": true,
 "has_size": false,

```

    "key": "radar:github:Lutra11/LTABC-KM",
    "kind": "radar",
    "languages": [],
    "match": {
      "idf_coverage": 0.4391,
      "matched_fields": [
        "description"
      ],
      "matched_tokens": [
        "assignment"
      ],
      "missing_tokens": [
        "credit",
        "training"
      ],
      "phrase_fields": [],
      "query_coverage": 0.3333,
      "ranking_algorithm": "bm25f_v3",
      "reason": "1 of 3 unique query tokens matched across fields; IDF coverage 0.44",

```

□
Build · DeepSeek V4 Flash DeepSeek

D:\projects\radar-test
187.3K (19%) · \$0.04 **ctrl+p** commands

Figure 5: A candidate record with a repository link for inspecting the implementation. Paper and dataset links are missing from the record.

```
"source": "GitHub",
"source_id": "mdkamranalam/credixa",
"updated_at": null,
"url": "https://github.com/mdkamranalam/credixa"
},
{
  "categories": [
    "benchmark"
  ],
  "description": "Long-horizon agentic tasks require large language models (LLMs) to iteratively retrieve, integrate, and maintain dispersed information across multi-turn interactions, but preserving all interaction histories leads to a continuously growing working context. Recent proactive context management methods allow models to edit their own working context with specialized tools, yet they still face three key limitations: (1) a limited toolset restricted to search, deletion, and summarization, with no support for global planning, long-term memory, and adaptive compression; (2) inefficient exploration that treats context management actions uniformly despite their heterogeneous impacts on final outcomes; and (3) coarse-grained credit assignment that assigns the final trajectory-level reward to all intermediate context editing actions during RL. To bridge these gaps, we introduce ContextPilot, a proactive context management framework for long-horizon agentic reasoning. Our approach systematically augments the toolset with planning, long-term memory, and soft context offloading tools. We further propose an RL method tailored for context management, which uses context and entropy variation to identify critical editing decisions for branch sampling and estimates action-level advantages from all branched trajectories that pass through the corresponding context editing action. Experiments on long-context QA and deep search tasks show that ContextPilot achieves stronger performance with a more compact working context, consistently outperforming existing baselines across various base models and benchmarks. Code is available at https://github.com/Tencent/ContextPilot.",
  "has_dataset": false,
  "has_paper": true,
  "has_repo": true,
  "has_size": false,
  "key": "radar:hugging face papers:2608.28476",
  "kind": "radar",
  "languages": [],
  "match": {
    "idf_coverage": 0.8392,
    "matched_fields": [
      "description"
    ],
    "matched_tokens": [
      "assignment",
      "credit"
    ]
  },
}
```

Build · DeepSeek V4 Flash DeepSeek

D:\projects\radar-test187.3K (19%) · \$0.04ctrl+p commands

Figure 6: A second candidate provides both paper and repository links. Its dataset link needs a follow-up check.

The agent supplemented catalog queries with web search to find related work described in different terms. It cross-checked the candidate sets and inspected the source evidence before selecting records for the comparison. Figure 7 shows the word matches and score components available for inspecting one retrieved candidate.

```

"kind": "radar",
"languages": [],
...
Click to expand

$ benchmark-radar search "credit assignment training" --scope radar --json
...
"query_coverage": 0.3333,
"ranking_algorithm": "bm25f_v3",
"reason": "1 of 3 unique query tokens matched across fields; IDF coverage 0.44",
"retrieval_score": 10.824384,
"score_components": {
  "bm25f": 10.824384,
  "name_bonus": 0.0,
  "phrase_bonus": 0.0
},
"modality": null,
"name": "deepvisionaiofficial/Automation_AI-Assignment-Checker",
"openness": null,
"published_at": "2026-08-10T05:56:17+00:00",
"publisher": "",
"rank": 6,
"recommended": false,
"score": 36.32,
"slug": null,
"snapshot_date": "2026-08-10",
"source": "GitHub",
"source_id": "deepvisionaiofficial/Automation_AI-Assignment-Checker",
"updated_at": null,
"url": "https://github.com/deepvisionaiofficial/Automation_AI-Assignment-Checker"
},
{
  "categories": [
    "benchmark",
    "agentic"
  ]
}

```

use internal rubrics to compute retrieval score

Build · DeepSeek V4 Flash DeepSeek

D:\projects\vradar-test 187.3K (10%) · \$0.04 ctrl+ip commands

Figure 7: A candidate record with query-word matches and retrieval-score components for the agent to inspect when judging relevance.

Figure 8 shows the resulting related-work table, which the contributor judged complete enough to act on.

#	Work	Published	Qwen base model(s)	Credit-assignment focus	Agentic benchmarks used	Link
1	SRPO: Self-Reflective Policy Optimization for Long-Horizon Reasoning	2026-08-25	Qwen3-8B (also Qwen3-1.7B / 32B scaling)	Reflection-conditioned dense token-level signals convert sparse terminal reward into training signal	AIIME24, WebShop, ALFWorld, SWE-Bench-Lite	https://arxiv.org/abs/2608.23493 · https://github.com/Galleons2029/SRPO
2	ContextPilot: Teaching Agents for Proactive Context Management via Fine-grained RL	2026-08-28	Qwen3-8B, Qwen3-14B	Coarse-grained → action-level credit assignment over branched trajectories during agentic RL	InfBench (InfiniteBench), NovelQA, LongMemEval, BrowseComp+	https://arxiv.org/abs/2608.28476 · https://github.com/Tencent/ContextPilot
3	SkillGate: Training In-Policy Skill Selection in Long-Horizon Agents	2026-08-21	Qwen3.5-9B (SFT checkpoint, RL init)	"Selector credit starvation": partitions token support into two credit channels (outcome vs. action-local advantage)	5 agentic benchmarks incl. SkillsBench, SWE (16-candidate skill slate)	https://arxiv.org/abs/2608.18852
4	CIPO: Contextual Information Policy Optimization for Search Agents	2026-08-06	Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct	Dense turn-level credit (EALR) to evidence-using reasoning actions, combined with outcome reward	HotpotQA, 2WikiMultiHopQA, MuSiQue, Bamboogle + 3 OOD	https://arxiv.org/abs/2608.06128
5	MoRSE: Task-Oriented Multi-Agent System with Mixture of Role-Subtask Experts	2026-08-10	Qwen3-4B-Instruct (one of three backbones; also Llama-3.1-8B, Gemma-4-31B)	Hierarchical GRPO with two-layer credit assignment (isolates expert vs. routing quality)	Code-generation benchmarks (multi-agent), held-out task domains	https://arxiv.org/abs/2608.09251

Figure 8: Summary table of recent work on credit assignment in agentic training, assembled during the session.

Figure 9 shows a comparison table from an earlier benchmark-design effort by one contributor. It covers ten related benchmarks across six design dimensions, with each cell read from a source paper. The contributor reported that assembling it took more effort than any part of that project except producing the benchmark data. Benchmark Radar retrieves candidates for this kind of comparison and exposes the matching words and fields for inspection; the comparison itself requires reading the source evidence.

Bench Name	End-to-End Tasks	Fine-Grained Eval	Researcher-Quality Eval	Data Generation	Multi-Harness Eval	#Tasks
MLE-Bench (Chan et al. 2025)	✗	✓	✗	Transfer&Compose	✓	75
MLGym-Bench (Nathani et al. 2025)	✗	✓	✗	Automatic	✗	13
EXP-Bench (Kon et al. 2025)	✓	✗	✗	Automatic	✓	461
ResearchCodeBench (Hua et al. 2026)	✗	✓	✗	Transfer&Compose	✗	212
MLR-Bench (Chen et al. 2026)	✓	✗	✗	Automatic	✓	201
PaperBench (Starace et al. 2025)	✗	✓	✗	Transfer&Compose	✗	8316
AstaBench (Bragg et al. 2025)	✓	✓	✗	Transfer&Compose	✓	2400+
InnovatorBench (Wu et al. 2025)	✓	✗	✗	Transfer&Compose	✗	20
AIRS-Bench (Lupidi et al. 2026)	✗	✓	✗	Automatic	✓	20
COMPOSITE-Stem (Waters et al. 2026)	✓	✓	✗	Manual	✗	70
ScienceBoard (Sun et al. 2025)	✗	✓	✗	Manual	✗	169

Figure 9: A prior-art comparison table assembled manually for an earlier benchmark-design effort. Each row is one related benchmark and each column one design dimension, all read from the source papers by hand.

This case records one workflow combining Benchmark Radar with web search; it has no measured time or accuracy comparison.³ The summary table and session evidence are available in the project’s public [issue #492](#).

C Reproducibility, Access, and Citation

This revision audits software commit 8f46bbf with a data cutoff of 2026-09-07. A fresh detached checkout passed the six required CI steps in order: linting, formatting checks, catalog normalization, classification, release construction, and tests. All 1,314 tests passed. The dated registry inputs remain the August snapshots registered in `data/leaderboard_snapshots.yml`; the September cutoff does not imply that those sources were recrawled on that day.

The data build normalizes the registry and model-report inputs into the shared catalog, then classifies discovery tracks and packages the catalog with checksums. Installed clients validate an update before activating it. The manuscript and its dated analysis files build independently of the software checkout. The project provides a DOI and Citation File Format record for citation [3, 12, 23].

Artifact	Canonical or permanent location
Archived paper (v0.9.0)	https://doi.org/10.5281/zenodo.22167102
Source code	https://github.com/ktwu01/benchmark-radar
Dashboard	https://benchmark-radar.org/
Discovery observations (JSON)	https://benchmark-radar.org/data/radar.json
Benchmark catalog	https://benchmark-radar.org/data/benchmark-index.json
RSS	https://benchmark-radar.org/feed.xml
Citation metadata	https://github.com/ktwu01/benchmark-radar/blob/main/CITATION.cff

Table 7: Access and citation artifacts for Benchmark Radar.

The census reads `site/data/benchmark-index.json`, the detail shards under `site/data/benchmarks/`, and the shared document and model registries. Discovery counts come from the rebuilt `site/data/radar.json` and dated snapshots. Model reports and score YAML files are normalization inputs, not a separate population for the paper’s findings.

³Jiayu Wang contributed this worked use case and its supporting evidence.

The paper repository contains `scripts/audit_catalog.py`, which validates IDs and counts, computes the census, and exports `catalog-data.tex` plus `evidence/catalog-audit.json`. The JSON includes every source-record key, measurement state, per-record model and document counts, citation IDs, and SHA-256 hashes of the inputs. The script checks score-observation uniqueness, record-to-shard identity, document-registry coverage, and reconciliation of scored and unscored records. Its `--check` mode compares a fresh calculation against the committed outputs. Figure 1 draws one linked dot per census record in source-key order.

To reproduce, check out software commit 8f46bbf, install its development dependencies, and run the six-step CI sequence. Run the software's `scripts/export_report_figure_data.py` and its `--check` mode to refresh `figure-data.tex`. From the paper checkout, run `python scripts/audit_catalog.py /path/to/software`, then repeat with `--check`. The README records the commands. Finally, run `make arxiv`, compile the extracted package, and inspect the rebuilt PDF. Normal paper builds use the committed data and do not refresh the evidence.

The DOI in Table 7 identifies the archived v0.9.0 paper. This manuscript describes software commit 8f46bbf at the cutoff above. The dashboard continues to change; citations of its live counts need a retrieval date.